Hashmap & Set Basics - Hashing
(Dictionary)

Dictionary: Stores data as key-value pairs.

- Key should be unique.
- Value can be non-unique.
- Keys should be Immutable
- Time to search, insert, update or delete a key-value pair - O(1)*

Population of every country: dict [str, int]

Number of states in every country: dict [str, int]

List of states of every country:

USA : California, Oregon, Washington, . . .

India : Delhi, Telangana, Karnataka, . . .

. . .

dict [str, list [str]]

Population of states in every country

USA : { California : 10M, Oregon : 4M, Washington : 6M }

Indian : { Delhi : 30M, Telangana : 60M, Karnataka : 40M }

dict [str, dict[str, int]]

<u>Sets</u>: Stores only unique values.

- Element should be unique.
- Element should be immutable.
- Search, insert, delete → O(1) operations.

- Both sets and dictionaries don't have indexing.
- Insertion order is not maintained in hashing.

Accessing the element is by the key.

Question: Given N elements and Q queries, find the frequency of the elements provided in the query

ex: arr = [2, 6, 3, 8, 2, 8, 2, 3, 8, 10, 6]
queries = [2, 3, 8, 5]
O/P = [3, 2, 3, 0]

Brute Force Approach

- for every query, iterate on the array to get the frequency of the element.

TC = O(N*Q) SC = O(1)

Optimised:

- Create a frequency map of the elements in the array.
- for each query, lookup the frequency map

```
def countQueries(arr, queries):
    # Create FM.
    fm = {}    # N space
    for ele in arr:    # N iterations.
        if ele in fm: fm[ele] += 1
        else: fm[ele] = 1
    # Answer queries
    output = []
    for ele in queries:    # Q iterations
        ans = fm.get(ele, 0)
        output.append(ans)
    return output.
```

TC = O(Q+N) SC = O(N)

ex: arr = [2, 6, 3, 8, 2, 8, 2, 3, 8, 10, 6]

queries = [2, 3, 8, 5]

fm = {2: 3, 6: 2, 3: 2, 8: 3, 10: 1}

Question: Given a list of integers, find the number of unique elements.

ex: arr = [1, 2, 1, 3, 2]

O/P = 3

arr = [3, 3, 3]

O/P = 1

```
def countUnique(arr):
    #return len(set(arr))
    #or
      s = set()
    for ele in arr:
        s.add(ele)
    return len(s)
```

TC = O(N) SC = O(N)

Question: Given a stream of integers, find the first non-repeating integer.

ex: arr = [500, 200, 200, 404, 500, 403]

O/P = 404

{500: 2, 200: 2, 403: 1, 404: 1}

Approach

- Create FM.
- Go over FM and return element with freq=1.

```
def firstNonRepeating(arr):
    # create FM.
    fm = {}
    for ele in arr:
        fm[ele] = fm.get(ele, 0) + 1
    # Iterate on fm.
    for key in fm:    # flawed.
        if (fm[key] == 1) return key
    return -1.
```

Flaw - Hashmap doesn't follow insertion order.

```
def firstNonRepeating(arr):
    # create FM.
    fm = {}
    for ele in arr:        # N iterations
        fm[ele] = fm.get(ele, 0) + 1
    # Iterate on arr.
    for key in arr:        # N iterations.
        if (fm[key] == 1)  return key
    return -1.
```

TC = O(N) SC = O(N)

Question: Given a list of integers, find the integers appearing more than N times.

ex: arr = [1, 5, 1, 2, 1, 5] N = 2

O/P = [1]

Brute Force Approach

- For every element, iterate over the array and count its frequency.
- If freq > N, then add to ans.

TC = $O(N^2)$ SC = $O(1)$

Optimised

- Create fm.
- Iterate over fm to get elements with freq > N

[1, 2, 3, 1, 2]

fm = {1: 2, 2: 2, 3: 1}

```
def moreThanNOccur(arr, N):
    # create FM.
    fm = {}
    for ele in arr:
        fm[ele] = fm.get(ele, 0) + 1
    # Iterate on fm.
    output = []
    for key in fm:
        if (fm[key] > N): output.append(key)
    return output.
```

TC = O(N) SC = O(N)